

# High-Throughput LDPC-Decoder Architecture Using Efficient Comparison Techniques & Dynamic Multi-Frame Processing Schedule

Sachin Kumawat, Rahul Shrestha, *Graduate Student Member, IEEE*, Nikunj Daga, and Roy Paily, *Member, IEEE*

**Abstract**—This paper presents architecture of block-level-parallel layered decoder for irregular LDPC code. It can be reconfigured to support various block lengths and code rates of IEEE 802.11n (WiFi) wireless-communication standard. We have proposed efficient comparison techniques for both column and row layered schedule and rejection-based high-speed circuits to compute the two minimum values from multiple inputs required for row layered processing of hardware-friendly min-sum decoding algorithm. The results show good speed with lower area as compared to state-of-the-art circuits. Additionally, this work proposes dynamic multi-frame processing schedule which efficiently utilizes the layered-LDPC decoding with minimum pipeline stages. The suggested LDPC-decoder architecture has been synthesized and post-layout simulated in 90 nm-CMOS process. This decoder occupies 5.19 mm<sup>2</sup> area and supports multiple code rates like 1/2, 2/3, 3/4 & 5/6 as well as block-lengths of 648, 1296 & 1944. At a clock frequency of 336 MHz, the proposed LDPC-decoder has achieved better throughput of 5.13 Gbps and energy efficiency of 0.01 nJ/bits/iterations, as compared to the similar state-of-the-art works.

**Index Terms**—Data-rate/throughput, IEEE 802.11n (WiFi) wireless-communication standard and very-large scale-integration (VLSI) design, LDPC codes, LDPC-layered decoding.

## I. INTRODUCTION

LOW-DENSITY parity-check (LDPC) codes are widely regarded for their excellent error-correction capabilities with near optimal bit-error rate (BER) performance and high flexibility in hardware realization [1]. These powerful features have made this code a popular choice for several wireless-communication standards like IEEE 802.11n (WiFi), IEEE 802.16e (WiMAX), IEEE 802.15c (WPAN), IEEE 802.3 (10 GBASE-T), DVB-S2 and ITU G.hn [2], [3]. Such widespread applications have led into the design of efficient LDPC decoding algorithms and their hardware implementations.

Min-sum algorithms (MSA) with offset are preferred over more robust sum-product algorithms because of their reduced

hardware complexity though at the expense of degraded BER performance. Two main scheduling schemes are used for decoding irregular-LDPC code in hardware: flooding and layered decoding schemes. Flooding schedule is utilized in [4]–[6] for its low-memory intensive implementations and data independency for easy division of the data path. Whereas, layered scheduling converges in almost half the number of iterations when compared to the flooding schedule [7]. Reported works like [8]–[12] efficiently use this property resulting in multi-gigabit decoders. However, layered schedule suffers from severe data dependencies which prevent very high-speed implementations. On the other side, inter-standard reconfigurability has attracted attention of researchers leading to works like [13]. They use shuffled decoding to reduce hardware complexity and support modern standards like IEEE 802.11n, IEEE 802.16e and ITU G.hn. Multi-mode reconfigurable architectures in [14] and [15] have the flexibility to switch between LDPC and turbo decoding-process. Condo *et al.* [14] uses different processing cores for LDPC and turbo decoding with shared memory to store messages for both the processes. While Sun and Cavallardo [15] have designed single architecture to process both LDPC and turbo codes by proposing a unified algorithm. Weiner *et al.* [6] and Flatresse *et al.* [16] studied the use of fully-depleted silicon-on-insulator (FD-SOI) technology resulting in low power solutions which opened up a new dimension at the technology level.

These contributions have elevated the performance of LDPC decoder in terms of power consumption, design area and throughput. However, there still exists a need to boost the achievable throughput from the state-of-the-art value to conceive an architecture that is compatible for next-generation wireless-communication systems. With rapidly growing demands of higher data-rates, wireless industries have predicted the benchmark throughput beyond 3 Gbps for next-generation standards [17]. Thereby, this work focuses on the design of high-throughput reconfigurable LDPC-decoder which supports multi-gigabit data-rate and the contributions are as follows:

- 1) A block-parallel layered-decoding scheme [10] utilizing the offset min-sum algorithm (O-MSA) has been used for the design of LDPC decoder. A major challenge for block parallel-architecture of LDPC decoder is to find the required two-minimum values for O-MSA from multiple inputs in single cycle. Thereby, we have proposed an efficient-comparison technique to be used in the construction of high-speed circuits for extracting the minimum values with smaller number of pipeline stages. This resulted in lower latency and good frequency scaling.
- 2) This work proposes dynamic multiple-frame processing for LDPC-decoding algorithm to remove the pipeline bubbles and minimize the memory overheads.

Manuscript received September 27, 2014; revised January 12, 2015; accepted January 27, 2015. Date of publication April 03, 2015; date of current version April 28, 2015. This paper was recommended by Associate Editor X. Zhang.

S. Kumawat, N. Daga and R. Paily are with the Indian Institute of Technology Guwahati, Guwahati, India (e-mail: s.kumawat@iitg.ernet.in; n.daga@iitg.ernet.in; roypaily@iitg.ernet.in).

R. Shrestha is with the Center for VLSI and Embedded System Technology (CEST), International Institute of Information Technology-Hyderabad (IIIT-H), Hyderabad-500032, Telangana, India (e-mail: rahul.shrestha@iiit.ac.in).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TCSI.2015.2403032

- 3) Using these proposed techniques, architecture of LDPC decoder is conceived for IEEE 802.11n (WiFi) wireless-communication standard [2]. The BER performance analysis of this decoder for various code-rates has been performed in additive-white Gaussian-noise (AWGN) channel environment. Synthesis and post-layout simulation of the suggested LDPC-decoder is carried out using 90 nm-CMOS process and it can be reconfigured for various code-rates and wide range of block lengths those are compliant to WiFi wireless-communication standard.

The remainder of this paper is organized as follows. In Section II, brief discussions on the formulation of LDPC-layered decoding and challenges faced in this work are carried out. Section III presents the proposed techniques on efficient-comparator designs and rejection based circuits for more number of inputs. In Section IV, reconfigurable architecture of LDPC decoder is presented and Section V includes BER performance evaluation of the suggested LDPC decoder, VLSI-design details and comparison with the reported works. Finally, this paper is concluded in Section VI.

## II. LDPC LAYERED DECODING: FORMULATION & CHALLENGES

Several variants of the layered MSA exist but its basic structure can be summarized as follows. Most of the wireless communication standards specify their parity-check matrix (PCM) in quasi-cycle low-density parity-check (QC-LDPC) form, as shown in Fig. 1. Such PCM is constructed using a  $Z \times Z$  identity sub-matrix called block, which is rotated or shifted by an amount specified by a shift matrix  $h$ . Let  $h(c, l)$  specifies the shift coefficient for  $c^{th}$  block column and  $l^{th}$  block layer in the PCM. Similarly, let  $M_{c,l}$  be the exclusive information sent from variable nodes to the corresponding check nodes;  $E_{c,l}$  be the extrinsic information sent from check nodes to the variable nodes in  $l^{th}$  block layer and  $c^{th}$  block column in vector form. Consider  $P_c$  to be the vector of posterior information or log-likelihood ratios (LLRs) for the variable nodes in  $c^{th}$  block column. Now if  $n$  is the number of sub-iterations and  $I_{max}$  is the maximum number of iterations, the message updates can be expressed as:

$$[M_{c,l}^k]^{h(c,l)} = [P_c^{k-1}]^{h(c,l)} - E_{c,l}^{k-1} \quad (1)$$

$$E_{c,l}^k = \Psi \left( [M_{c',l}^k]^{h(c',l)}, c' \in A_l / \{c\} \right) \quad (2)$$

$$[P_c^k]^{h(c,l)} = [M_{c,l}^k]^{h(c,l)} + E_{c,l}^k \quad (3)$$

where  $l = \{1, 2, 3 \dots n\}$  and  $k = \{1, 2, 3 \dots I_{max}\}$ . Here  $k$  denotes the iteration number and  $A_l$  is the set of variable nodes connected to check nodes in  $l^{th}$  layer. Vectors with superscript  $h(c, l)$  are circularly rotated by that amount and the vectors  $P_c^{(0)}$  are initialized to the received channel values and  $E_{c,l}^{(0)}$  are initialized to zero. In the case of layered decoding approach, the QC-LDPC PCM is divided into block of layers which can be considered as sub-codes. For each sub-iteration or layer processing, the parity check constraints for only that layer are processed. The messages  $M_{c,l}$  are immediately updated after processing each layer and are used for the next layer, as illustrated by (1). The function  $\Psi(\cdot)$  in (2) denotes the check node processing and, for O-MSA, it can be expressed as:

$$\Psi(x_i, x_i \in S) = \lambda_S \cdot Q_S \quad (4)$$

where

$$\lambda_S = \prod_{x_i \in S} \text{sign}(x_i), \quad Q_S = \max(\min(x_i) - \delta, 0) \quad (5)$$

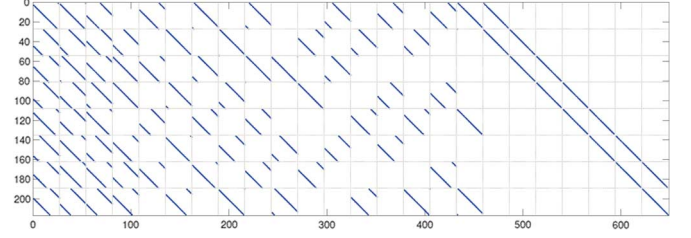


Fig. 1. An IEEE 802.11n rate 2/3, 648 frame length QC-LDPC parity check matrix. The boxes represent rotated sub-matrices or blocks.

where  $S$  is a set over which inputs (i.e.,  $x_i$ ) to the function are defined and  $\delta$  is the offset which is a correction measure to improve the performance of MSA.

One of the major challenges of this formulation is to produce messages for the next layer in the subsequent clock cycle. For block parallel implementation considered here, this includes finding the minimum from multiple inputs, which are 24 in the case of IEEE 802.11n, removal of offset and generation of the messages for all the neighboring variable nodes in single clock cycle. This increases critical-path delay and pipelining becomes necessary to cut it down. But the data dependencies from (1)–(3) result in a feedback which introduces multiple pipeline bubbles equal to  $S - 1$  where  $S$  is the number of stages. Thereby, block interlacing [18] and multi-layer processing [19] utilize these pipeline bubbles. As a result, block interlacing method processes two frames together in interlaced pattern while the multiple layer processing modifies the layered decoding scheme to process two layers with the same set of  $M_{c,l}$  messages, simultaneously, thus increases the number of iterations required for convergence. Gunnam *et al.* [8] propose some measures based on memory bypassing and pre-fetching along with reordering of the PCM to reduce the data dependencies.

We therefore propose a technique to process multiple frames simultaneously to fill up the pipeline bubbles. Additionally, we have suggested data pre-fetching method to remove memory read-write latencies, as majority of the memory accesses are sequential rather than random. On the other side, the number of pipeline stages and the number of frames should be minimized because messages for each frame have to be stored separately which increases the memory overheads. In this work, the number of frames to be processed is decided on the basis of frame length which makes it dynamic multi-frame processing, elaborated in latter sections.

## III. FAST MINIMUM COMPUTATIONS

The check node processing in layered O-MSA schedule includes computations of two most minimums out of the incoming messages (denoted by  $\min1$  and  $\min2$ ), index of  $\min1$ , sign bits for each check-node output, and the offset processing. Thereby, high-speed check-node units (CNUs) with low-area occupation are needed as they are the major contributor of logic resources in the decoder architecture. Most taxing sub module in CNU is the computations of  $\min1 - \min2$  and  $\min1$  index. Therefore, this paper proposes low-complexity techniques for comparing magnitudes of two numbers in two's complement (2C) and sign magnitude (SM) formats for column and row layered processing respectively. Then, we suggest efficient rejection-based schemes to compute  $\min1 - \min2$  out of more than 2 inputs.

### A. Proposed Techniques for Efficient-Comparator Design

Conventional comparator takes two SM values and uses most-significant bits (MSBs) to compare. Further, lower-order bits are

**Inputs:**  $a$  and  $b$  in two's complement form (Big-Endian)  
**Output:** Flag  $f$ :  $f = 0$  for  $|a| < |b|$  and  $f = 1$  for  $|a| > |b|$   
1:  $msbCheck \leftarrow a[DW] \oplus b[DW]$   
2: **for**  $i \leftarrow 1$  **to**  $DW - 1$  **do**  
3:  $temp[i] \leftarrow a[i] \oplus \overline{msbCheck}$   
4: **end for**  
5:  $signCheck \leftarrow cla(temp, b[DW - 1 \text{ to } 1], \overline{msbCheck})$   
6:  $f \leftarrow signCheck \oplus b[DW]$

Fig. 2. Suggested technique for comparison of two 2C numbers.  $DW$  is the data width and  $cla(x, y, z)$  stands for CLA with  $x$  and  $y$  as inputs and  $z$  as the initial carry.

TABLE I  
FPGA IMPLEMENTATION RESULTS: 2'S COMPLEMENT AND SIGN MAGNITUDE COMPARATORS

| Algorithm     | 2's complement (2C) |       | Sign Magnitude (SM) |       |
|---------------|---------------------|-------|---------------------|-------|
|               | Conv.               | Prop. | Conv.               | Prop. |
| Area (# LUTs) | 15                  | 7     | 5                   | 3     |
| Delay (nS)    | 7.320               | 7.467 | 7.015               | 6.729 |

compared if the higher-order bits are equal and this results in  $O(n)$  logic-path delay for  $n$ -bit inputs. We have suggested a carry look-ahead adder (CLA) based comparison scheme that results in  $O\{\log(n)\}$  path delay. The effectiveness of using CLA for comparison has been reported in previous works like [20]. Fig. 2 presents the suggested technique for comparing two 2C numbers.

The basic idea of this method is to add two numbers with different signs and check the sign bit of the result. If both the inputs have the same sign, which is denoted by  $msbCheck$  in Fig. 2, one of the inputs (i.e.,  $a$  here) is inverted by taking one's complement (1C) by XORing with  $msbCheck$ , otherwise it is passed unaltered. Then, this value is added with other input using CLA where initial carry input to CLA serves as the increment of one required for 1C to 2C conversion. Note that computation of the complete sum is not required, rather only MSB of the result is needed which can be efficiently carried out using a CLA in  $O\{\log(n)\}$ . Finally, the flag is generated as 0 if both the inputs are positive and  $signCheck$  is 0 or if both are negative and  $signCheck$  is 1. Flag will also be 0 if they have different signs with  $a$  being positive and  $signCheck$  is 1 or  $a$  is negative and  $signCheck$  is 0. Otherwise the flag becomes 1 and this is ensured by taking XOR of  $signCheck$  with sign of the other input  $b$ . The proposed algorithm is particularly efficient in column layered or block serial processing of LDPC codes. There the messages are processed serially inside the CNU and can be directly compared in 2's complement form.

We implemented a conventional 2C comparator which consists of two 2C to SM converters and a conventional  $O(n)$  comparator. Table I shows post place-&-route results for the implementation of conventional and suggested 2C magnitude comparators on field-programmable gate-array (FPGA) board (Virtex-IV, xc4vlx160 device). The data width is 5 bits and both the units generate a flag rather than propagating the minimum input to the output. It can be readily observed that the number of logic units is significantly reduced with marginal increase in combinatorial delay.

For the row layered block parallel scheduling as in our case, multiple inputs are processed in parallel. Therefore it makes sense to convert the 2C inputs to SM once and propagate the inputs in hierarchical computations. We can therefore optimize the above technique for SM inputs using the fact that both the inputs would be of positive sign. Fig. 3 presents the simplified technique. This results in a classical circuit that subtracts the two unsigned inputs and checks the sign of the output computed by cla. Table I also presents the implementation results for SM comparison units.

**Inputs:**  $a$  and  $b$  in sign magnitude form (Big-Endian)  
**Output:** Flag  $f$ :  $f = 0$  for  $a < b$  and  $f = 1$  for  $a > b$   
1:  $temp \leftarrow a[DW - 1 \text{ to } 1]$   
2:  $f \leftarrow cla(temp, b[DW - 1 \text{ to } 1], '1')$

Fig. 3. Modified technique for magnitude comparison of two sign magnitude numbers.

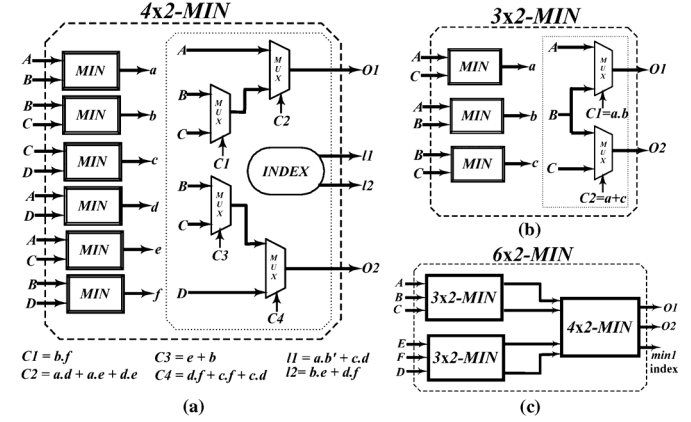


Fig. 4. (a) Rejection based four-input two-output comparator circuit. (b) Rejection based three-input comparator circuit. (c) Hierarchical architecture of six-input comparator based on rejection scheme.

The conventional comparator here is just a single  $O(n)$  comparator. It can be seen that the scheme again results in a lower number of logic units with lower delay. Therefore, we use the SM comparator units to develop fast multi-input circuits with low area utilization.

### B. Rejection Based Multi-Input Circuits

To find the minimum out of multiple inputs which require propagation of data; generally, tree structure (TS) based circuits are considered the fastest with degraded area. Such circuits find  $min1$  and  $min2$  at each stage and combine them until only one pair of the least two minimums are left [21], [22]. This results in  $\log(n)$  stages for  $n$  inputs and low delay ( $\log(n)$  units in the critical path). Sorting based approaches (XS) are another popular choice which consist of  $\log(n)$  levels when  $min1$  is computed. However, to find both  $min1$  and  $min2$ , another  $\log(n)$  units are needed for  $min2$  computation [8], [21], [23].

To achieve a decoder frequency of 110 MHz (see Section IV-A) in FPGA for higher number of inputs as 24 in our case, pipelining becomes necessary. XS circuits suffer from a feedback which cannot be efficiently pipelined to improve upon speed. On the other hand, though TS schemes can be pipelines, they suffer from large area due to more logic resources. Therefore, we have suggested a rejection based scheme (RS) for tree structures in which non-useful inputs are rejected and the rest (least two) are propagated forward. We save on  $min1$  and  $min2$  computations at each stage resulting in area occupations similar to XS and delay performance approaching TS circuits.

For  $n$  inputs, parallel  $nC_2$  comparisons are carried out and flags are generated for controlling a multiplexer (mux) layer. The mux network rejects all but two inputs which are the least two minimums in unknown order. Fig. 4(a) shows an architecture for 4-input comparator along with logic for mux-control and  $min1$  index bits generation. Each "MIN" block is based on Fig. 3 which outputs a flag denoted by  $a, b, \dots, f$ . The mux control logic is a function of the six flags. Six-variable K-map reduction can be used to obtain the logic as given in Fig. 4(a). Furthermore, the  $min1$  index bits,  $I1$  (LSB) and  $I2$  (MSB), can also be found as a function of the 1-bit flags as shown in Fig. 4(a). For  $n = 2^k$ , we

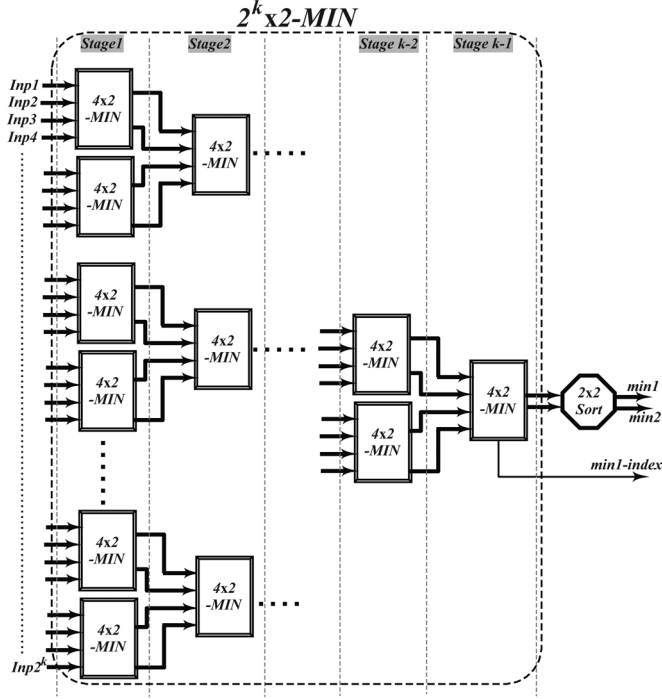


Fig. 5. Suggested general-architecture of  $2^k$ -input comparator based on rejection scheme.

TABLE II  
COMPARATOR CIRCUITS FOR DIFFERENT NUMBER OF INPUTS

| # Inputs | Sorting Based [8] |       |       | Rejection Based (RS) |       |       | H'6   |
|----------|-------------------|-------|-------|----------------------|-------|-------|-------|
|          | 3                 | 4     | 6     | 3                    | 4     | 6     |       |
| # A      | 3                 | 4     | 7     | 3                    | 6     | 15    | 12    |
| # M      | 4                 | 7     | 14    | 2                    | 4     | 10    | 8     |
| CP (A,M) | (3,4)             | (3,4) | (5,7) | (1,1)                | (1,2) | (1,3) | (2,3) |

use the radix-4 unit to construct a TS comparator, rejecting half of the inputs at each stage as shown in Fig. 5. At any stage  $s$ , the  $min1$  index is calculated as:

$$idx(msb)_s = b_s \cdot e_s + d_s \cdot f_s \quad (6)$$

$$idx(rest)_s = \overline{idx(msb)_s} \cdot idx1_{s-1} + idx(msb)_s \cdot idx2_{s-1} \quad (7)$$

where the subscript denotes attributes for that stage and  $idx1$  and  $idx2$  are indices from previous stage radix-4 units. Finally a simple two-input sorter, which comprises of one comparator and two multiplexers can be used to find  $min1 - min2$  out of the least two. For many practical applications, the number of inputs may not be a power of two. For such cases, we can use mixed radix circuits. Fig. 4(b) presents the rejection based scheme for 3 inputs. The mux control logic for radix-3 is a function of the flags  $a$ ,  $b$ ,  $c$ , and can be easily obtained as:

$$I1 = b \cdot \bar{c}, I2 = a \cdot c$$

Table II presents a comparison between modules with different number of inputs based on the survival based scheme for selection sorting presented by Gunnam *et al.* [8] and the proposed rejection based approach. They use comparison flags and  $min1$  index to extract survivors from the previous levels after  $min1$  computation. Out of the extracted survivors,  $min2$  is computed. In the Table II, “A” and “M” stands for number of comparators/CLAs

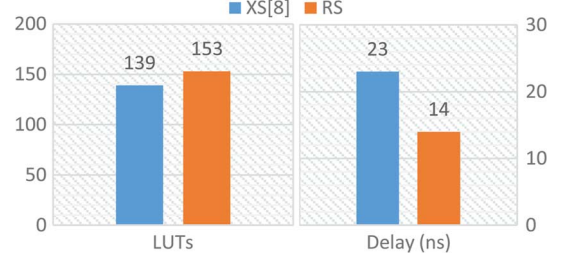


Fig. 6. Comparison of FPGA post place-&-route results for eight-input comparators.

and 2:1 multiplexers respectively. CP stands for critical path in terms of logic units “A” and “M.”

It can be noted that the proposed circuits are efficient in terms of critical path delay and have no feedback when compared to XS approach. For example, for four input circuits, suggested scheme results in one comparator/CLA and two multiplexers in the critical path as compared to three and four, respectively, for [8]. However, the number of flags ( ${}^nC_2$ ) increase quickly with an increase in  $n$  by a power of two which makes the mux-control logic more complex. For example, for six-input module with  ${}^6C_2 = 15$  flags, the mux-control logic is a function of 15 variables and the resulting data path will become very complex. Therefore, our approach does not scale properly with the number of inputs. Hence, we use multiple three-input and a four-input unit to realize a hierarchical six-input (H'6) architecture. Fig. 4(c) show the H'6 architecture. Table II also presents the estimated attributes for H'6. The  $min1$  index calculation remains the same for the first  $3 \times 2 - MIN$  unit but we need to modify it for the second one by simply giving the 4th input to H'6 as its last input as shown in Fig. 4(c) and changing the index equations as:

$$I1 = a \cdot c, I2 = a + c$$

For the  $4 \times 2 - MIN$  unit, the msb calculation is modified as:

$$I3 = (\bar{a} + \bar{b}) \cdot (b \cdot e + d \cdot f)$$

and propagated according to (7).

To compare the proposed approach with XS, we implemented the survival based scheme in [8] on FPGA for 8 inputs. For the suggested approach, we used three four-input modules which are stacked together along with a two-input sorter. Fig. 6 presents the results suggesting significant reduction in delay with marginal increase in area.

Table IV suggests area and delay results for a comparator and 2-input mux implemented using 90 nm CMOS process. Since we use the optimized comparator module based on Fig. 3, area occupied by the comparator was found to be similar to a 2-input mux for 5-bit data width. Therefore reduction in number of muxs also have a significant effect on the area which is a key feature of the rejection-based scheme. Additionally, given the longer critical path for a comparator, minimum number of comparators are ensured in the suggested rejection based scheme.

We also implemented the proposed circuits on 90 nm CMOS process with 0.9 V supply voltage in worst case delay conditions to compare with other existing schemes. The comparison is summarized in Table III. Reference [23] presents improvements over XS approach in [21], so we compare with XS in [23] and TS in [21]. It can be seen that the proposed scheme achieved better gate count than the improved XS in [23]. Due to lack of information, proper delay scaling is not carried out but theoretically RS performed at similar speeds as [23]. It also performs significantly

TABLE III  
COMPARISON OF RS WITH EXISTING XS AND TS SCHEMES: DW (DATA WIDTH), T (CMOS-TECHNOLOGY NODE), A (DESIGN AREA), D (DELAY), GC (GATE COUNT), P (AREA-DELAY PRODUCT)

|          | Proposed RS (DW=5, T=90nm) |                                             |      | XS [23] (DW=5, T=65nm) |                                             |      | XS [21] (DW=6, T=90nm) |                                             |      | TS [21] (DW=6, T=90nm) |                                             |    |
|----------|----------------------------|---------------------------------------------|------|------------------------|---------------------------------------------|------|------------------------|---------------------------------------------|------|------------------------|---------------------------------------------|----|
|          | A( $\mu\text{m}^2$ )       | D(ns)<br>P( $\text{nm}^2 \cdot \text{nS}$ ) | GC   | A( $\mu\text{m}^2$ )   | D(ns)<br>P( $\text{nm}^2 \cdot \text{nS}$ ) | GC   | A( $\mu\text{m}^2$ )   | D(ns)<br>P( $\text{nm}^2 \cdot \text{nS}$ ) | GC   | A( $\mu\text{m}^2$ )   | D(ns)<br>P( $\text{nm}^2 \cdot \text{nS}$ ) | GC |
| $n = 8$  | 1019.2                     | 2.6<br>0.0026                               | 211  | -                      | 2.3<br>-                                    | 287  | 8985                   | 2.93<br>0.0263                              | 391  | 8808                   | 1.57<br>0.0138                              | -  |
| $n = 16$ | 2387.2                     | 3.9<br>0.0093                               | 492  | -                      | 3.4<br>-                                    | 678  | 21412                  | 3.68<br>0.0788                              | 951  | 19605                  | 2.15<br>0.0421                              | -  |
| $n = 32$ | 5130.4                     | 5.3<br>0.0272                               | 1055 | -                      | 4.4<br>-                                    | 1386 | 49025                  | 4.84<br>0.2372                              | 2127 | 41226                  | 2.73<br>0.1125                              | -  |
| $n = 24$ | 6370                       | 5.0<br>0.0318                               | 1406 | -                      | -<br>-                                      | -    | -                      | -<br>-                                      | -    | -                      | -<br>-                                      | -  |

TABLE IV  
COMPARATOR VS. MULTIPLEXER PERFORMANCE

| Logic Unit  | Area ( $\mu\text{m}^2$ ) | Delay (ns) |
|-------------|--------------------------|------------|
| Comparator  | 32.144                   | 0.441      |
| Multiplexer | 35.280                   | 0.143      |

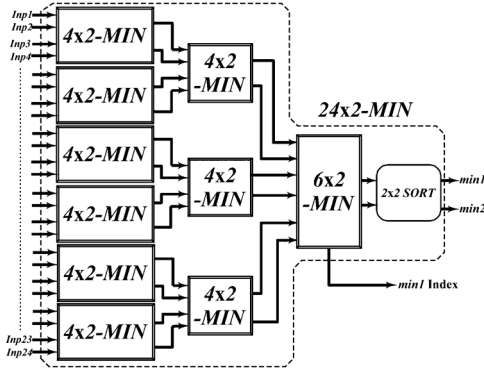


Fig. 7. 24-input minimum ( $\text{min1} - \text{min2}$  and  $\text{min1}$  index) computation architecture.

better in terms of area when compared to designs in [21]. Furthermore, RS achieves the best area-delay product results among the reported works.

For the cases where  $n$  is not a power of 2, such as IEEE 801.11n (WiFi) standard which defines PCMs with 24 block columns, we can use mixed radix circuits with hierarchical stacking. Fig. 7 presents the architecture for  $n = 24$  where we have used multiple modules of the proposed radix-4 circuits and a six-input module in hierarchical fashion along with the two-input sorter. To use the H'6 module in hierarchical structure, the index in  $3 \times 2 - \text{MIN}$  units are modified to propagate previous stage indices as:

$$\text{idx}(\text{msb})_s = a_s, \text{idx}(\text{rest})_s = \bar{a}_s \cdot \text{idx}1_{s-1} + a_s \cdot \text{idx}2_{s-1}$$

While for the internal  $4 \times 2 - \text{MIN}$  unit, the index calculations are customized depending on  $n$ . For  $n = 24$ , we modify it as:

$$\begin{aligned} \text{tmp} &= b_s \cdot e_s + d_s \cdot f_s \\ \text{idx}(\text{msb})_s &= \text{idx}(\text{msb})_{s2} \\ \text{idx}(\text{rest})_s &= \text{tmp} \cdot \text{idx}_{s1} + \text{tmp} \cdot \text{idx}_{s2} \end{aligned}$$

where  $\text{idx}_{s1}$  and  $\text{idx}_{s2}$  are indices from the internal  $3 \times 2 - \text{MIN}$  units at the same stage  $s$ . Moreover, dividing the data path just before the six-input module makes it efficiently pipelined as the data path is effectively divided in half resulting in even higher operating clock frequency. Table III also gives implementation results

for the pipelined 24 input RS architecture. The delay mentioned is combinational delay considering combinatorial data path.

#### IV. MULTI-RATE BLOCK-PARALLEL LDPC DECODER

WiFi standard specifies up to four MIMO transmitters capable of transmitting at 150 Mbps each. This results in a data throughput of 600 Mbps or user throughput of at least 300 Mbps for rate-1/2 codes. Therefore to fully support WiFi standard, the target throughput is 600 Mbps. Block-serial or column layered approach is usually more energy efficient and reconfigurability is less complex; however, it requires 24 cycles for a single iteration for WiFi standard. While block-parallel or row-layered approach processes 24 inputs together in less number of clock cycles (12 clock cycles) in a single iteration. This makes it suitable for the design of high-throughput decoders. To reduce the reconfiguration complexity, we have used a look-up table based technique for the suggested block-parallel architecture.

##### A. Decoder Architecture

For a block-parallel CNU that is compliant to WiFi standard, 24 inputs are read per cycle and then  $\text{min1}$ ,  $\text{min2}$ ,  $\text{min1}$  index, and sign computations are performed. Since there are maximum 12 layers which have to be processed for rate-1/2 codes, with the maximum number of ten iterations ( $I_{\text{max}}$ ), the minimum required clock frequency for the decoder is calculated as 108 MHz for FPGA implementation. We have designed three pipeline stages in CNU to achieve a target frequency of 110 MHz. Fig. 8(a) presents the suggested CNU architecture.

The CNU processing flow starts by reading 24 messages through the first multiplexer network,  $\text{MUX-NTW-1}$ . It implements 2-input muxs for each of the CNU inputs to replace them with a maximum value if, according to the PCM, the CNU is not connected to the variable-node unit (VNU) corresponding to that input. This method allows filtering of varying number of non-neighboring VNU messages based on corresponding 24 flags from  $\text{PCM-FLAG ROMs}$  [Fig. 8(b)] dynamically for different row degrees. The filtering is followed by conversion of 2C values to SM by  $\text{ABS}$  block. The conversion is required for utilizing the low complexity comparison scheme of Fig. 3 in the  $\text{MIN COMPARATOR BLOCK}$ , which assumes all inputs to be positive. Simultaneously, a XOR tree,  $\text{SIGN CALCULATION UNIT}$ , computes the signs for CNU outputs. The next step is offset correction for the two minimums as required by O-MSA (5), carried out by subtractors as shown in Fig. 8(a). After offset removal, if the results are negative (value underflow), they are replaced by an all zero value, as governed by (5). Sign bit of the results act as control signals for the multiplexers used for the

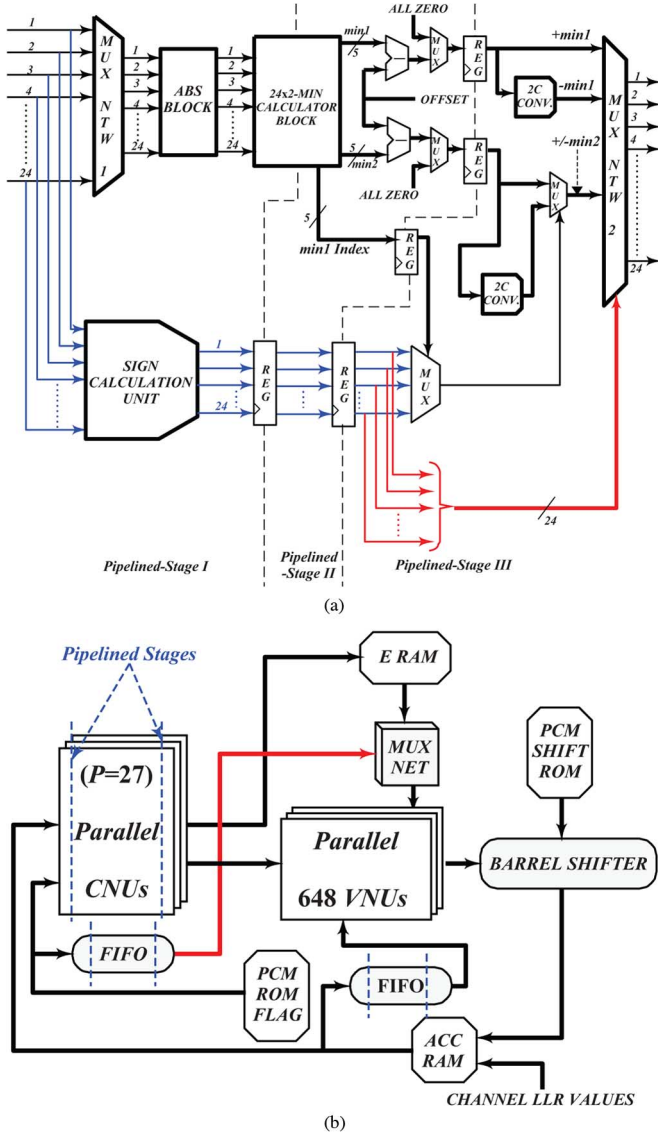


Fig. 8. (a) Proposed block-parallel CNU architecture. (b) Proposed reconfigurable multi-rate layered-LDPC decoder-architecture.

purpose. Since signs for all the outputs are already computed along with the  $\min1$ , only  $+\min1$ ,  $-\min1$  and  $\min2$  with the computed sign are stored to reduce memory requirements. Finally, a second multiplexer network,  $MUX-NTW-2$ , expands the incoming 3 values into 24 CNU outputs. For each output, one mux selects between  $+\min1$  and  $-\min1$  based on the sign bit calculated by the XOR tree and a second mux selects between the propagated  $\min1$  and  $\min2$  based on the  $\min1$  index. The 3 pipeline stages, dividing the CNU data flow, are carefully set so as to uniformly partition the data path inside the decoder.

Fig. 8(b) shows the proposed top-level architecture of reconfigurable block-parallel layered LDPC-decoder. The dashed lines represent pipeline stages. A parallelization factor of  $P(≤ Z)$  is chosen by using  $P$  CNU in parallel, each processing one parity constraint for a particular row layer. A set of subtractors and adders perform the operations of (1) and (3), respectively, and together constitutes the VNUs which work in serial fashion. For the decoder, we chose  $P = 27$  CNU that correspondingly require 648 VNUs to support parallel processing. For each cycle,  $P \times 24 = 648$  messages are exchanged between CNU and VNU groups. The longer frames of 1296 and 1944 are handled in two

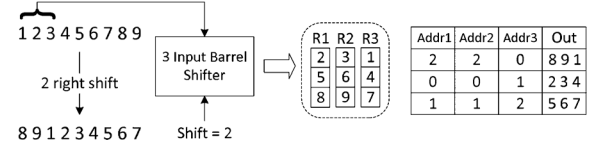


Fig. 9. Circular shifting of longer frames using barrel shifter and proper memory addressing. “R” are the RAMs and “Addr” are their corresponding read addresses. “Out” is a group of three messages out of nine-length vector in the shifted form.

and three cycles respectively. For the layered schedule, we can use only one set of shifters to achieve the required cyclic shifts by modifying the shift coefficients ( $h(c, l)$ ) as follows. If  $x$  is an entry in the shift matrix  $h$  and  $y$  represents an entry for the layer below it, then the required shift is calculated as:

$$y_{new} = (Z + y - x) \bmod Z$$

At the start of a new frame,  $x$  is initialized to  $Z$ . The barrel shifters can process 648 messages per cycle but circular shifting of the longer frames is not straightforward. For example, a vector of length 10 cannot be rotated using two five-input shifters. Neither can it be accomplished in two cycles using one five-input shifter directly. Thus, we have proposed a memory addressing based technique which utilizes two and three cycle latencies for longer frames to address this issue.

Consider a three-input shifter and three random-access memories (RAMs), as shown in Fig. 9. To perform two right-shifts of nine-length vector, we rotate its first three elements rightwards by two units using the shifter and store them in the RAMs, as shown in Fig. 9. It is repeated for the next three elements in the next cycle and so on until the complete vector is stored in that manner. Now during the memory-read cycles, data is read by setting proper addresses for each RAM. Fig. 9 shows the address sequences required for the task and the corresponding output sequence. The output sequence has been rotated by two and can be read in groups of three for parallel processing. In general for a vector of  $P \times 24$  elements, a shift of  $s$  can be achieved using an  $n$  input barrel shifter by rotating subset of  $n$  elements by  $(s \bmod n)$ . The read addresses needs to be calculated before the first access, after which the access is circularly sequential in nature.

$ACC$ -RAMs are true-dual port RAMs used to accumulate the rotated  $M_{c,l}$  messages after each sub-iteration, as shown in Fig. 8(b). Proper addressing of the RAMs is done to read the messages in correct shifted order. Similarly,  $E$ -RAMs store output of the CNU ( $E_{c,l}$ ) in the compressed form as  $+\min1$ ,  $-\min1$  and  $+\min2$  after each sub-iteration. These messages are required to generate  $M_{c,l}$  messages for the next layer as given by (1). The 12 PCMs are stored in  $PCM$ -SHIFT ROMs while  $PCM$ -FLAG ROMs store the flag information used by CNUs for each PCM. The address generators form the controller that ensure data pre-fetching, bypassing and correct order of message flow. FIFOs act as latency synchronizers between the CNU inputs and outputs.

The controller works in four major states. Processing of an incoming frame begins by populating  $ACC$ -RAMs with channel LLR values constituting the initialization state. The second state comprises of first rotation of CNU input messages according to  $h(c, l)$  coefficients. The messages are bypassed to barrel shifter via VNUs by adding and subtracting zeros and stored back to  $ACC$ -RAMs in shifted manner. The third state starts the decoding process by reading CNU messages from  $ACC$ -RAMs.  $P \times 24 = 648$  messages are read by the  $P$  CNU and pipelined processing of the data begins. The emerging CNU to VNU messages out of CNU are simultaneously stored in the  $E$ -RAMs and read by 648

VNUs in parallel. CNU inputs for the next layer are computed in VNUs by adding to previous CNU inputs and subtracting previous layer CNU outputs from it as denoted by (3) and (1) respectively. The messages are then shifted according to PCM coefficients for the next layer and stored in *ACC-RAMs* again which marks the completion of one sub-iteration. A decoding iteration is completed when all the layers are processed. Each layer is processed for a pre-set number of iterations  $I_{max}$  before decoding stops. The last controller state takes decoding decisions based on signs and flushes out the output for current frame.

### B. Reconfiguration and Multi-Frame Processing

Status of the currently-decoded code is stored in the form of addresses of look-up tables containing information of the number of layers and the number of frames to process. We have stored all the PCMs in a single ROM to save memory and the base address (i.e., starting address) for each PCM is also stored in a look-up table. Therefore to reconfigure the decoder to process a frame with different code rate, simply the address for number of layers and base-address table need to be updated which is carried out by the controller. This results in processing the required number of sub-iterations/layers and correct PCM-shift coefficients from the ROM. Similarly to switch to a code with different frame length, the address to number of frames and base-address table are updated.

As discussed before, the pipeline stages introduce idle latency cycles. Therefore, we store and process multiple frames together to fill up the pipeline bubbles and efficiently utilize the layered schedule. Memory write takes two clock cycles introducing two memory latency cycles in addition to the two-cycle logic-pipeline latency. Memory-read cycles are avoided using data pre-fetching as the read addresses can be pre-computed. Hence, there is a 4 stage bubble while processing a 648 length frame or part of longer frames. Since for QC-LDPC codes, only parity checks within a layer have independent computations that can be done in parallel, multiple frames can be processed to utilize the idle stages. The number of frames are decided dynamically based on the frame size. Fig. 10 shows the dynamic multi-frame processing. The symbol  $f_p^{[l]}$  represent frame number ( $f$ ), frame part or the clock cycle number for longer frames ( $p$ ) and the PCM layer/row number ( $l$ ). For 648 length frame, we bypass the accumulation RAMs as the shifter can rotate it in one cycle itself, avoiding the two-memory write cycles. Hence, we process three frames for 648 length codes as shown in Fig. 10. For 1296-length frames, it takes two cycles to process one frame and circular shifting also require multiple cycles. Therefore, it requires three frames, each occupying 2 stages so that the bubbles are not introduced. It is to be noted that in this case, multiple message vectors have to be stored in the memory. Similarly for 1944-length frames, it takes three cycles to process a single frame and thus two frames are enough to occupy all the 5 stages in the processing flow, as shown in Fig. 10.

## V. PERFORMANCE ANALYSIS, VLSI DESIGN, AND COMPARISON OF RESULTS

High-level behavioral simulation of the suggested LDPC decoder has been carried out using binary-phase shift-keying (BPSK) modulation in AWGN-channel environment for WiFi wireless-communication standard. This simulation includes BPSK-modulated transmission of  $10^{12}$  bits via AWGN channel; subsequently, the noisy channel outputs are demodulated and then decoded using proposed LDPC decoder for each of the

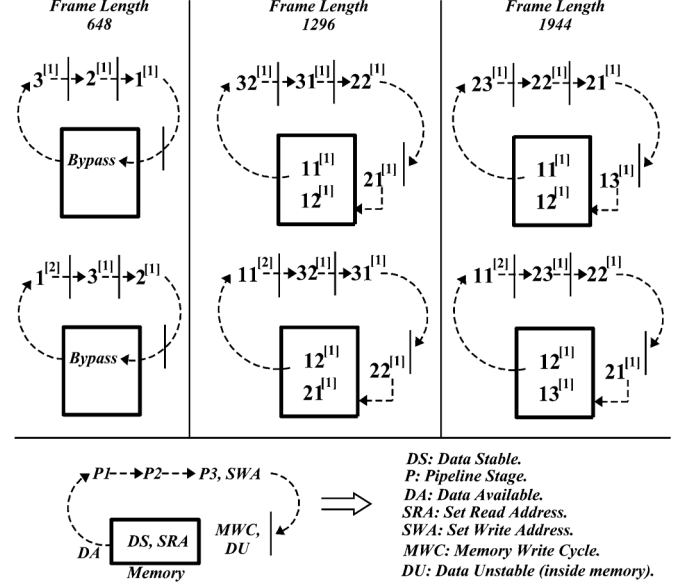


Fig. 10. Proposed multi-frame processing for efficient utilization of pipeline. Here  $f_p^{[l]}$  denotes  $f^{th}$  frame's  $p^{th}$  part under  $l^{th}$  layer processing. For 648 length frames, each frame is processed in a single cycle and has no parts. Refer to the legend at the bottom for meaning of different stages in the processing.

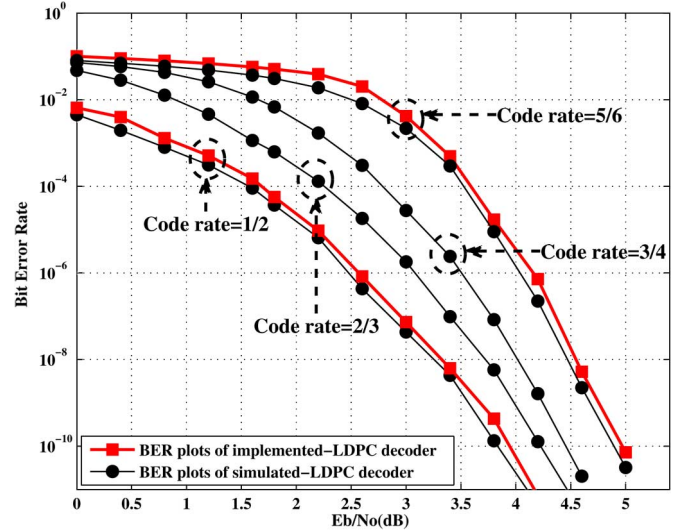


Fig. 11. BER performance of the suggested LDPC-decoding algorithm for various code-rates and a block length of 1944 compliant to WiFi standard.

$E_b/N_0$  values. Thereafter, the BER performance of such LDPC decoder for ten decoding iterations and a maximum block-length of 1944 bits, which is compliant to WiFi standard, has been plotted in Fig. 11. Additionally, the suggested decoder can decode code with multiple code-rates that ranges from low-effective code-rate of 1/2 to high-effective code-rate of 5/6. It is to be noted that for modern wireless-communication systems, the full range of code-rates between these corners must be supported by the decoder architecture [24]. Thereby, high-level simulation has been performed for WiFi-compliant code-rates like 1/2, 3/4, 2/3, and 5/6. It can be observed that the suggested LDPC decoder achieves a BER of  $10^{-9}$  at  $E_b/N_0$  values of 3.56 dB and 4.66 dB for low-effective code-rate of 1/2 and high-effective code-rate of 5/6, respectively, as shown in Fig. 11. Similarly, the suggested decoder attains a BER of  $10^{-9}$  at 3.98 dB and 4.25 dB for the code-rates of 2/3 and 3/4 respectively.

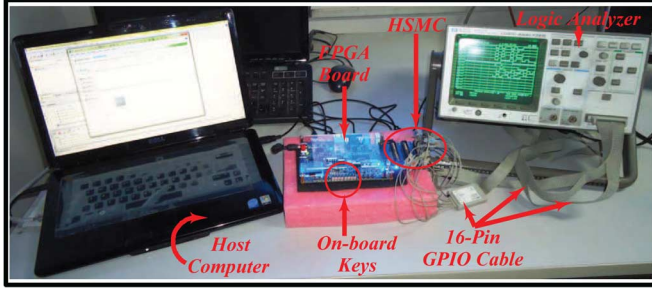


Fig. 12. Test setup for the hardware testing of suggested LDPC decoder using host computer, FPGA, and logic analyzer.

TABLE V  
RESULTS AND COMPARISON FOR FPGA IMPLEMENTATIONS

|                      | Proposed     |              | [10]’2011 |
|----------------------|--------------|--------------|-----------|
| Quantization (bits)  | (6,2)        | (5,2)        | (6,2)     |
| Code Rate            | 1/2-5/6      | 1/2-5/6      | 1/2       |
| Block Length         | 648-1944     | 648-1944     | 672       |
| Slices               | 35105        | 28145        | 27003     |
| LUTs                 | 67204        | 52565        | 45409     |
| FPGA Family          | Virtex-4     | Virtex-4     | Virtex-4  |
| FPGA Device          | xc4vlx160    | xc4vlx160    | xc4vls200 |
| Clock Freq. (MHz)    | 116          | 121          | 335       |
| Data Through. (Mbps) | 617.1-1808.4 | 617.1-1808.4 | 1644      |
| User Through. (Mbps) | 308.6-1507   | 308.6-1507   | 822       |

We have hardware-implemented the proposed LDPC decoder on FPGA board (Virtex-IV, xc4vlx160 device) with (5,2) and (6,2) fixed-point quantization where 2 represents the number of bits for fractional precision. Channel LLR values with fixed-point representation are derived from the high-level behavioral model used in BER simulation and are stored in on-board random-access memory (RAM) of FPGA. These LLRs are stored in FPGA memory by host computer where the BER simulations were carried out, as shown in Fig. 12. Thereafter, configuration file (in.bit format) of the suggested decoder-architecture is loaded on FPGA board by host computer via universal-serial bus (USB) cable. One of the on-board keys has been used as enable signal to on-board memory and implemented decoder. The output of this on-board decoder is interfaced with logic analyzer using 160 pins high-speed mezzanine card (HSMC) which has data transfer speed of 3.125 Gbps, as shown in Fig. 12. Subsequently, 16-pin GPIO cable has been used to connect HSMC and the logic analyzer. Thereby, for each  $E_b/N_0$  value, entire channel LLR values are stored using on-board memory and these values are fed to implemented LDPC decoder on triggering high the enable signal using on-board key. The decoded LLR values are displayed on the screen of logic analyzer and the decoded bits are stored in host computer as a file (in.txt format) which is used for computing the BER value. Thus, BER values are computed for different  $E_b/N_0$  values for the code-rates 1/2 as well as 5/6 and are plotted in Fig. 11. These BER plots are derived from fixed-point architecture of suggested LDPC decoder with quantization (5,2) and it has shown minor degradation of  $\approx 0.1$  dB for both the code-rates of 1/2 and 5/6 at a BER of  $10^{-9}$ . Table V shows the results obtained for this FPGA implementation with (5,2) and (6,2) fixed-point quantization. It can be observed that the decrement in slice and LUT utilizations of 19.82% and 21.78%, respectively, have occurred with the reduction of one bit. In case of registers and savings in memory, gains of 14.49% and 7.36%, respectively, can be observed. Therefore, we have used five-bit quantization because

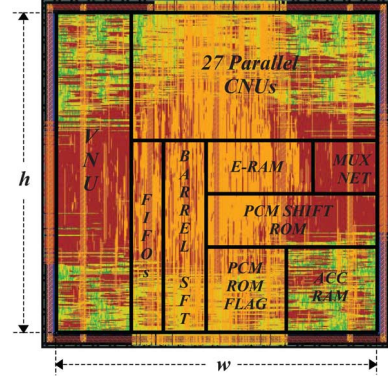


Fig. 13. Chip layout of proposed LDPC decoder with core dimension of  $(h \times w) = (2.38 \text{ mm} \times 2.21 \text{ mm})$ .

TABLE VI  
ACHIEVABLE THROUGHPUTS (Mbps) OF PROPOSED-LDPC DECODER FROM FPGA IMPLEMENTATION AND ASIC DESIGN FOR ALL THE BLOCK LENGTHS AND CODE RATES OF WiFi STANDARD

| Code-Rate | FPGA |      |      | ASIC |      |      |
|-----------|------|------|------|------|------|------|
|           | 648  | 1296 | 1944 | 648  | 1296 | 1944 |
| 1/2       | 617  | 622  | 612  | 1723 | 1737 | 1709 |
| 2/3       | 918  | 929  | 915  | 2564 | 2595 | 2554 |
| 3/4       | 1215 | 1234 | 1215 | 3392 | 3446 | 3392 |
| 5/6       | 1795 | 1837 | 1808 | 5010 | 5128 | 5048 |

this value resulted in adequate BER performance as well as appreciable hardware efficiency and the suggested decoder operates at maximum clock frequency of 121 MHz. We have compared the FPGA-implementation results with the decoder reported in [10], as shown in Table V. It can be observed that with competitive resource utilization and lower clock speed, we have achieved a higher user-throughput and support multiple codes. The data throughput ( $T_{data}$ ) of LDPC decoder can be computed as

$$T_{data} = \left( \frac{f \times l_f}{I_{max} \times l \times (Z/P) \times f + l_c} \right) \times f_{clk}$$

where  $f$  is the number of frames processed simultaneously,  $l_f$  is the frame length,  $l$  is the number of layers present in the code and  $f_{clk}$  is the running clock frequency of the decoder. The number of latency cycles  $l_c$  can be calculated as

$$l_c = (S - 1) + l \times (Z/P) + 2$$

where  $S-1$  are the pipeline latency cycles,  $l \times (Z/P)$  are the initial cyclic-shift rotation cycles and the last two cycles are for first-time memory-write wait. Thus, the user throughput can be calculated as  $T_{user} = r_c \times T_{data}$  where  $r_c$  denotes code-rate. For a 648 length and rate-1/2 code, we process three frames at 120 MHz in FPGA resulting in a data throughput of 617.1 Mbps.

The proposed LDPC decoder has been synthesized and post-layout simulated in 90 nm 1P9M-Low-K CMOS process with 0.9 V supply. Gate-level netlist of our design has been generated using SYNOPSIS-Design-Compiler tool and it possesses 18 logic-levels consuming 509 k and 4.36 k hierarchical and leaf cells respectively. Post-synthesis simulations and static-timing analysis of this netlist have been carried out using SYNOPSIS-Verilog-Compiler-Simulator and SYNOPSIS-Prime-Time tools respectively. Thereby, this decoder has a critical path delay of 2.98 nS. This verified netlist is imported in Cadence-SOC-Encoder tool for backend-design process with six-metal layers. Standard cells are placed on the core area with a cell-utilization of 79.2% and power grids as well as stripes are inserted on the

TABLE VII  
COMPARISON OF PROPOSED LDPC DECODERS WITH REPORTED WORKS

|                                | Prop.     | [13] <sup>†</sup> | [25] <sup>†</sup> | [26] <sup>†</sup> | [27] <sup>†</sup> | [28] <sup>†</sup> | [29] <sup>§</sup> | [30] <sup>§</sup> | [31] <sup>§</sup> | [32] <sup>†</sup> |
|--------------------------------|-----------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|
| Process (nm)                   | 90        | 90                | 90                | 90                | 90                | 90                | 65                | 65                | 130               | 180               |
| Codes                          | 12        | 114+12+7          | 12                | 114+12            | 114               | 114+12            | 114               | 114               | 12                | 114+12            |
| Flexibility                    | 11n       | 16e/11n/G.hn      | 11n               | 11n/16e           | 16e               | 16e/11n           | 16e               | 16e               | 11n               | 16e/11n           |
| Clock Freq. (MHz)              | 336       | 400               | 250               | 450               | 400               | 300               | 400               | 400               | 500               | 100               |
| Bit Quant.                     | 5         | 6                 | 8                 | –                 | 7                 | 6                 | 6                 | 7                 | 6                 | 8                 |
| Iteration                      | 10        | 8–12              | 1–7               | 10                | 8–12              | 20                | 10–15             | 15                | 5                 | 10                |
| Throughput (Gbps)              | 1.71–5.13 | 0.34–1.96         | 0.67              | 1.0               | 0.07–0.2          | 0.21              | 0.08–0.67         | 0.09–0.50         | 0.27–1.35         | 0.17              |
| Gate Count                     | 509K      | 377K              | –                 | –                 | 171K              | 947K              | –                 | –                 | 99.9K             | 198K              |
| Area (mm <sup>2</sup> )        | 5.2       | 5.53              | 3.67              | 3.5               | 0.68              | 6.22              | 1.337             | 0.55              | 1.85              | 10.12             |
| Scaled Area (mm <sup>2</sup> ) | 5.2       | 5.53              | 3.67              | 3.5               | 0.68              | 6.22              | 2.56              | 1.05              | 0.89              | 2.53              |
| Power (mW)                     | 451.3     | 517.7             | 171.1             | 410               | 63.2              | 528               | –                 | –                 | –                 | 411               |
| Ener. Effn. (nJ/b/iter.)       | 0.01      | 0.02              | 0.04              | 0.04              | 0.03              | 0.12              | –                 | –                 | –                 | 0.24              |

†: Post-Layout Simulation Results; §: Synthesis Results.

core. Then, clock-tree synthesis and timing optimizations are performed. Power routing, signal routing and insertion of leaf cells are carried out. Finally, post-route static-timing analysis as well as optimization is accomplished. Decoder layout has a core area of 5.19 mm<sup>2</sup> with 509 k gates. Using the standard delay files (.sdf) and the netlist generated from the layout, post-layout simulation is carried out and the design operates at 335.57 MHz. It has consumed a maximum power of 451.28 mW while decoding QC-LDPC code of 1944 block length and at 3/4 code-rate. Chip layout of the implemented LDPC decoder at 90 nm-CMOS process is shown in Fig. 13. Based on the results obtained from such application-specific integrated-circuit (ASIC) synthesis and post-layout simulation, we have obtained the achievable throughputs of the suggested LDPC-decoder for various code-rates and block lengths those are compliant to WiFi standard, as shown in Table VI. It shows that the maximum achievable throughputs of the decoder when implemented on FPGA and ASIC are 1.84 Gbps and 5.13 Gbps respectively.

We have compared the post-layout simulated results of the proposed LDPC-decoder with reported works, as shown in Table VII. Among the reported works, multi-standard LDPC decoder based on shuffled decoding proposed by Ueng *et al.* [13] has reported the highest throughput of 1.96 Gbps. The Suggested LDPC-decoder has achieved a throughput of 5.13 Gbps which is 62% better than the highest reported value of [13]. However, [13] supports multiple wireless-communication standards albeit single standard is supported by the proposed decoder-architecture. The proposed work also compares slightly better in area with [13]. The higher gate count figure for the proposed decoder is due to utilization of registers as memory cells during synthesis. Similarly, our work has achieved 87% better throughput than that of the state-of-the-art work [25], which is compliant to WiFi wireless-communication standard, and has a power reduction of 62%. Comparison has also been carried out with the reported works of lower (65 nm) and higher (130 and 180 nm) CMOS processes [29]–[32] in Table VII. It shows that the suggested decoder-architecture has achieved better throughput among all the reported works with nominal degradation in design area and power consumption. On the other side, we have achieved better energy efficiency of 0.01 nJ/bits/iterations which is 50% better than the best reported value from [13].

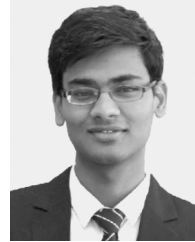
## VI. CONCLUSION

Efficient circuits for two, three, four and six inputs comparisons to address the issue of fast  $\min1 - \min2$  computations in a block parallel-layered LDPC-decoder architecture have been presented. The proposed architecture for 24 inputs was found to be faster and with small area overhead. The LDPC-decoder architecture proposed can fully support the WiFi wireless-communication standard in FPGA with a clock speed of 120 MHz. Post-layout simulation outcomes of the suggested LDPC decoder in 90 nm-CMOS process has shown very-high achievable throughput of 5.13 Gbps. Additionally, better energy-efficiency of 0.01 nJ/bits/iterations was achieved with minor degradation in design area and power consumption. The techniques suggested in this work can be applied to any LDPC decoder design to achieve multi-gigabit architecture for next-generation wireless-communication systems.

## REFERENCES

- [1] R. Gallager, “Low-density parity-check codes,” *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, 1962.
- [2] IEEE Std 802.11n-2009 (Amendment to IEEE Std 802.11-2007 as Amended by IEEE Std 802.11k-2008, IEEE Std 802.11r-2008, IEEE Std 802.11y-2008, and IEEE Std 802.11w-2009), 2009, Tech. Rep..
- [3] IEEE Std 802.16e-2005 and IEEE Std 802.16-2004/Cor 1-2005 (Amendment and Corrigendum to IEEE Std 802.16-2004), 2006, Tech. Rep..
- [4] F. Verdier and D. Declercq, “A low-cost parallel scalable FPGA architecture for regular and irregular LDPC decoding,” *IEEE Trans. Commun.*, vol. 54, no. 7, pp. 1215–1223, 2006.
- [5] M. Weiner, B. Nikolic, and Z. Zhang, “LDPC decoder architecture for high-data rate personal-area networks,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2011, pp. 1784–1787.
- [6] M. Weiner, M. Blagojevic, S. Skotnikov, A. Burg, P. Flatresse, and B. Nikolic, “27.7 A scalable 1.5-to-6 Gb/s 6.2-to-38.1 mW LDPC decoder for 60 GHz wireless networks in 28 nm UTBB FDSOI,” in *Proc. IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2014, pp. 464–465.
- [7] E. Sharon, S. Litsyn, and J. Goldberger, “Convergence analysis of serial message-passing schedules for LDPC decoding,” in *Proc. 6th Int. ITG-Conf. Source Channel Coding (TURBOCODING) and 4th Int. Symp. Turbo Codes & Related Topics*, 2006, pp. 1–6.
- [8] K. K. Gunnam, G. S. Choi, and M. B. Yeary, “A parallel VLSI architecture for layered decoding for array LDPC codes,” in *Proc. 20th Int. Conf. VLSI Design and 6th Int. Conf. Embedded Syst.*, 2007, pp. 738–743.

- [9] Z. Cui, Z. Wang, and X. Zhang, "Reduced-complexity column-layered decoding and implementation for LDPC codes," *IET Commun.*, vol. 5, no. 15, pp. 2177–2186, 2011.
- [10] S. Kim, G. E. Sobelman, and H. Lee, "A reduced-complexity architecture for LDPC layered decoding schemes," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 19, no. 6, pp. 1099–1103, 2011.
- [11] C.-W. Sham, X. Chen, W. M. Tam, Y. Zhao, and F. C. M. Lau, "A layered QC-LDPC decoder architecture for high speed communication system," in *Proc. IEEE Asia Pacific Conf. Circuits Syst. (APCCAS)*, 2012, pp. 475–478.
- [12] C. Yu, H.-S. Chuang, B.-S. Lin, P.-H. Cheng, and S.-J. Chen, "Improvement on a block-serial fully-overlapped QC-LDPC decoder for IEEE 802.11n," in *Proc. IEEE Int. Conf. Consum. Electron. (ICCE)*, 2014, pp. 446–447, IEEE.
- [13] Y.-L. Ueng, B.-J. Yang, C.-J. Yang, H.-C. Lee, and J.-D. Yang, "An efficient multi-standard LDPC decoder design using hardware-friendly shuffled decoding," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 60, no. 3, pp. 743–756, Mar. 2013.
- [14] C. Condo, M. Martina, and G. Masera, "VLSI implementation of a multi-mode turbo/LDPC decoder architecture," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 60, no. 6, pp. 1441–1454, 2013.
- [15] Y. Sun and J. R. Cavallaro, "A flexible LDPC/turbo decoder architecture," *J. Signal Process. Syst.*, vol. 64, no. 1, pp. 1–16, 2011.
- [16] P. Flatresse, B. Giraud, J. Noel, B. Pelloux-Prayer, F. Giner, D. Arora, F. Arnaud, N. Planes, J. Le Coz, O. Thomas, S. Engels, G. Cesana, R. Wilson, and P. Urard, "Ultra-wide body-bias range LDPC decoder in 28 nm UTBB FDSOI technology," in *Proc. IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, 2013, pp. 424–425.
- [17] D. Talbot, "A banner year for mobile devices," *MIT Technol. Rev., Commun. News*, Dec. 2012.
- [18] A. Darabiha, A. C. Carusone, and F. R. Kschischang, "Block-interlaced LDPC decoders with reduced interconnect complexity," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 55, no. 1, pp. 74–78, 2008.
- [19] Y. Sun, G. Wang, and J. R. Cavallaro, "Multi-layer parallel decoding algorithm and VLSI architecture for quasi-cyclic LDPC codes," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2011, pp. 1776–1779.
- [20] S. Veeramachaneni, M. K. Krishna, L. Avinash, P. S. Reddy, and M. Srinivas, "Efficient design of 32-bit comparator using carry look-ahead logic," in *Proc. IEEE Northeast Workshop Circuits Syst. (NEWCAS)*, 2007, pp. 867–870.
- [21] C.-L. Wey, M.-D. Shieh, and S.-Y. Lin, "Algorithms of finding the first two minimum values and their hardware implementation," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 55, no. 11, pp. 3430–3437, 2008.
- [22] L. G. Amarù, M. Martina, and G. Masera, "High speed architectures for finding the first two maximum/minimum values," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 20, no. 12, pp. 2342–2346, 2012.
- [23] Q. Xie, Z. Chen, X. Peng, and S. Goto, "A sorting-based architecture of finding the first two minimum values for ldpc decoding," in *Proc. IEEE 7th Int. Colloq. Signal Process. Its Appl. (CSPA)*, 2011, pp. 95–98.
- [24] Y. Sun and J. R. Cavallaro, "Efficient hardware implementation of a highly-parallel 3GPP/LTE-advance turbo decoder," *Integr., the VLSI J.*, vol. 44, p. 30517315, 2011.
- [25] C. Yu, H.-S. Chuang, B.-S. Lin, P.-H. Cheng, and S.-J. Chen, "Improvement on a block-serial fully-overlapped QC-LDPC decoder for IEEE 802.11n," in *Proc. IEEE Int. Conf. Consum. Electron. (ICCE-2014)*, pp. 446–447.
- [26] Y. Sun and J. R. Cavallaro, "A low power 1-Gbps reconfigurable LDPC decoder design for multiple 4G wireless standards," in *Proc. IEEE Int. SOC Conf.*, 2008, pp. 367–370.
- [27] Y. L. Wang, Y. L. Ueng, C. L. Peng, and C. J. Yang, "Processing-task arrangement for a low-complexity full-mode WiMAX LDPC codec," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 58, no. 2, pp. 415–428, Feb. 2011.
- [28] C.-H. Liu, C.-C. Lin, S.-W. Yen, C.-L. Chen, H.-C. Chang, C.-Y. Lee, Y.-S. Hsu, and S.-J. Jou, "Design of a multimode QC-LDPC decoder based on shift-routing network," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 56, no. 9, pp. 734–738, 2009.
- [29] T. Brack, M. Alles, T. Lehnigk-Emden, F. Kienle, N. Wehn, N.-E. L'Insalata, F. Rossi, M. Rovini, and L. Fanucci, "Low complexity LDPC code decoders for next generation standards," in *Proc. Design, Autom., Test Eur. Conf. Exhib. (DATE '07)*, Apr. 2007, pp. 1–6.
- [30] G. Gentile, M. Rovini, and L. Fanucci, "Low-complexity architectures of a decoder for IEEE 802.16e LDPC codes," in *Proc. Euromicro Conf. Digit. Syst. Design Archit., Methods, Tools (DSD)*, Aug. 2007, pp. 369–375.
- [31] K. Gunnam, G. Choi, W. Wang, and M. Yeary, "Multi-rate layered decoder architecture for block LDPC codes of the IEEE 802.11n wireless standard," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS 2007)*, pp. 1645–1648.
- [32] D. Bao, B. Xiang, R. Shen, A. Pan, Y. Chen, and X. Y. Zeng, "Programmable architecture for flexi-mode QC-LDPC decoder supporting wireless LAN/MAN applications and beyond," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 57, no. 1, pp. 125–138, 2010.



**Sachin Kumawat** received the B.Tech. degree in electronics and communication engineering along with a minor in computer science and engineering from the Indian Institute of Technology Guwahati, Guwahati, India in 2014.

He is currently working as a research and development software engineer in Chennai area, India. His research interest lies in the areas of embedded systems and VLSI-architecture design for various fields including digital communication, networks and computer vision.



**Rahul Shrestha** received the B.Eng. degree in telecommunication engineering from B.M.S College of Engineering, Bangalore, India, in 2008. He received the Ph.D degree in electronics and electrical engineering from Indian Institute of Technology Guwahati, Guwahati, India in 2014.

He is currently an Assistant Professor at the International Institute of Information Technology Hyderabad, Center for VLSI and Embedded Systems Technologies, Telangana, India. His research interests include VLSI design and implementation of high-speed digital architectures for wireless communication applications in ASIC/FPGA platform and the study of channel codes from algorithmic and implementation perspectives for high-throughput applications.



**Nikunj Daga** received the B.Tech. degree in electronics and communication engineering from Indian Institute of Technology Guwahati, Guwahati, India in 2014.

He is currently working as a technical associate in Bangalore, India. He has experience in embedded system design and authored a landscape analysis whitepaper for intellectual property of 2G-3G wireless communications. His research interest lies in VLSI design, digital communications, and computer networks.



**Roy Pailly** received the B.Tech. degree in electronics and communication engineering from College of Engineering, Trivandrum, India in 1990. He obtained the M. Tech. and Ph.D. from Indian Institute of Technology, Kanpur and Indian Institute of Technology, Madras in 1996 and 2004 respectively, in the area of semiconductor devices.

He is currently a Professor at the Department of Electronics and Electrical Engineering, and Head of the Centre of Nanotechnology, Indian Institute of Technology Guwahati, Guwahati, India. His research interests are VLSI circuits, MEMS, and devices.